

RentalApp

Peer-to-Peer Rental Marketplace

40535392

Justin Wylie

Submission date: 28 July 2026

GitHub repository

https://github.com/justinwylie033/RentalApp_40535392

1. Project overview

RentalApp is a compact Android marketplace in which community members list useful items, find nearby listings, request rentals, progress a controlled rental workflow, and leave verified reviews. The implementation deliberately completes the brief's Tier 1 and Tier 2 requirements and adds the State Pattern and automatic overdue detection from Tier 3.

Submission claim

Evidence-based position

The implementation completes the core marketplace, spatial, workflow, review, architecture, testing, and delivery requirements. The final GitHub Actions run measured 87.4% line coverage, exceeding the brief's 80% distinction testing threshold. The grade remains subject to marking and oral defence.

Technology stack

Area	Technology	Evidence
Client	.NET 10 / .NET MAUI	Android XAML, Shell, Secure Storage, GPS, address geocoding
API	ASP.NET Core 10	Minimal API endpoints, JWT bearer authentication, Problem Details
Data	EF Core 10	DbContext, migrations, repositories, Unit of Work
Spatial	PostgreSQL 16 + PostGIS	Geography point, SRID 4326, GiST, radius query
Quality	xUnit + Coverlet	89/89 passing tests; 87.4% line coverage
DevOps	Docker Compose + GitHub Actions	Healthy local services, test/coverage job, signed APK job
Environment	VS Code Dev Container	.NET 10 Noble image and C# / Docker extensions

Report map

Architecture 3-6 | Features 7-12 | Testing 13-14 | CI/CD 15 | Patterns 16-17 | AI use 18 |
References and verification 19

2. Architecture documentation

2.1 Component diagram

RentalApp component architecture

Dependencies flow through interfaces; business rules remain at the API boundary.

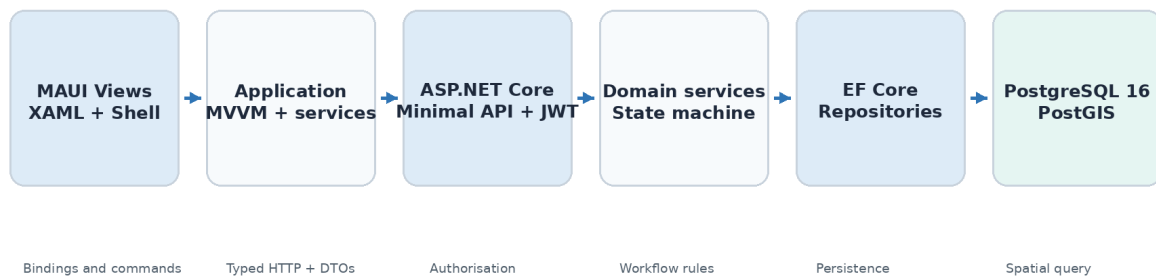


Figure 1. Component boundaries and dependency direction.

The mobile client never connects directly to PostgreSQL. Views bind to ViewModels, application services call the authenticated HTTP client, and the API owns security and business rules. EF Core repositories isolate persistence and spatial queries. This supports testability and a clear separation of concerns.

Design rationale

Interfaces keep Android concerns out of ViewModel tests, while the API remains the authoritative boundary even if a client bypasses the UI.

2.2 Database schema

Database schema

UUID keys, explicit relationships, spatial indexing, and review integrity.

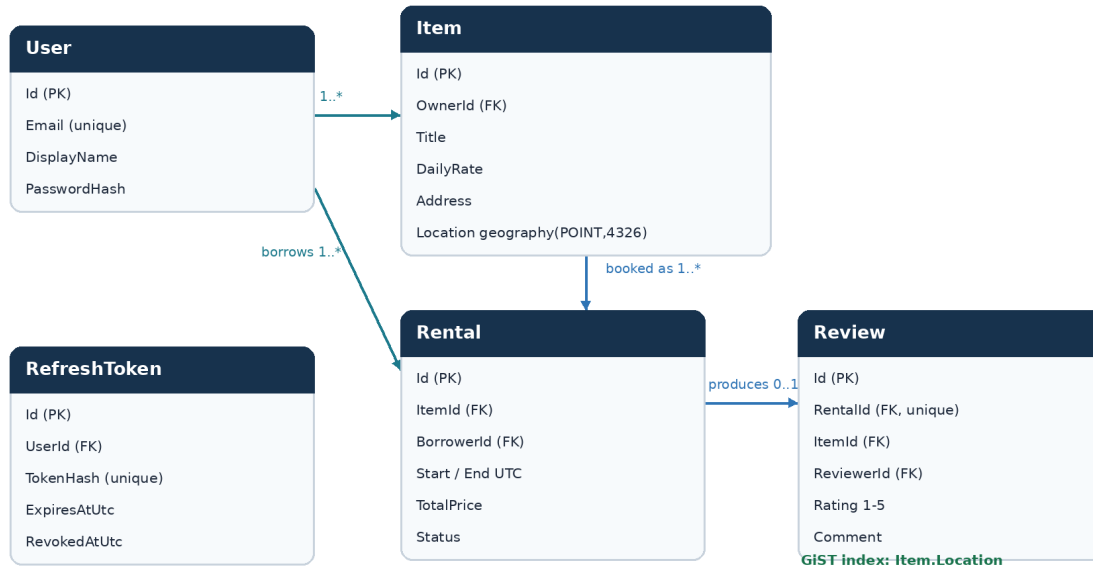


Figure 2. Core entities and integrity relationships.

Item.Address stores the readable collection address; the MAUI client forward-geocodes typed addresses or reverse-geocodes device GPS. Item.Location is mapped to geography (point, 4326), giving distance in metres. A GiST index supports the nearby predicate. Reviews have a unique RentalId and rating check constraint.

2.3 Rental request sequence

Rental request sequence

The API authenticates the caller and enforces booking rules before persistence.

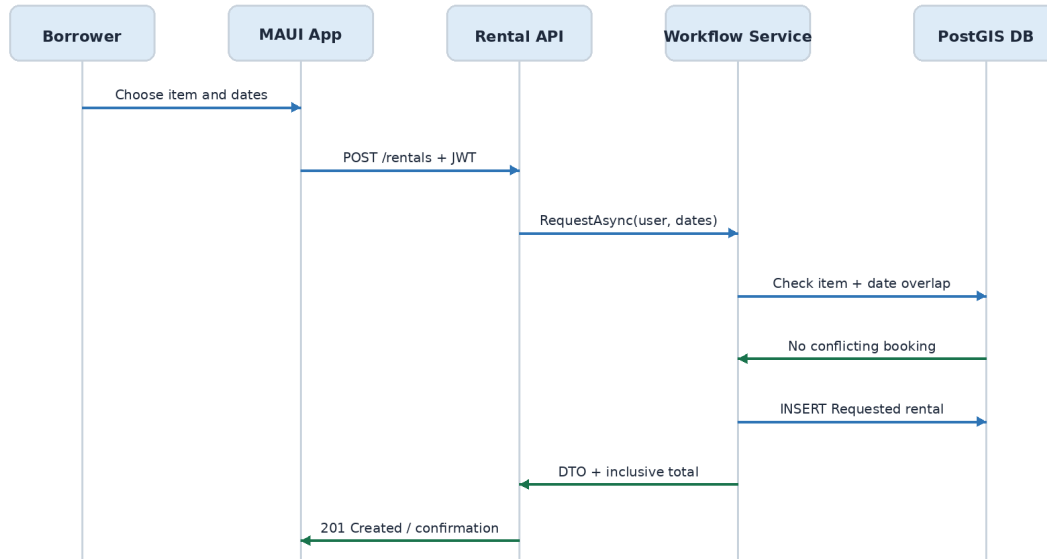


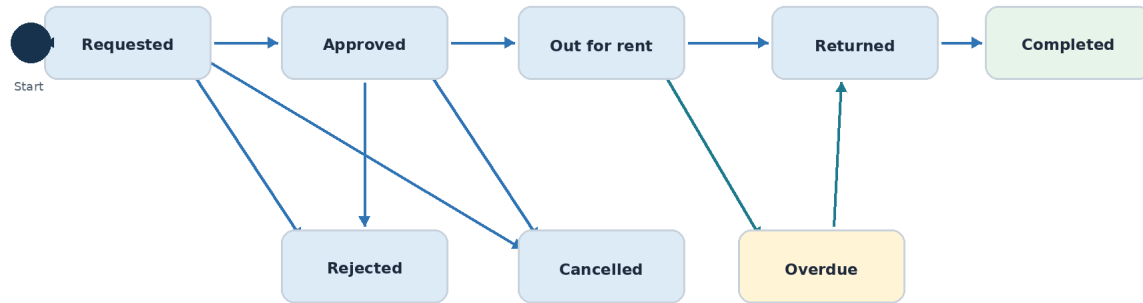
Figure 3. Successful rental request from the MAUI app to PostgreSQL.

RentalWorkflowService validates future dates, availability, self-rental, and inclusive overlap before calculating price and committing. The response includes the Requested status and total price. Exceptions are converted to consistent API errors and displayed by the ViewModel.

2.4 Rental state diagram

Rental state machine

State classes validate workflow transitions; the service validates the acting user's role.



Automatic transition: Out for rent -> Overdue when the due date passes.

Figure 4. Permitted workflow transitions implemented by state classes.

Each of the eight state classes declares its permitted successors. Relationship authorisation is intentionally separate: listing creators respond and confirm, while requesters cancel eligible requests and confirm return. The background overdue worker checks expired out-for-rent records and applies the same state machine.

3. Feature implementation checklist

Done	Requirement	Implementation evidence
YES	Registration and sign-in	Password hashing, JWT/refresh tokens, rate limiting, retry handling
YES	Authenticated client	Bearer injection, token rotation, Secure Storage, visible sign-out
YES	Item catalogue	Create, browse, details, creator edit, availability, My Listings
YES	Marketplace discovery	Search, sort, category filter, paging, review-count display
YES	Address entry	Forward geocoding, reverse GPS lookup, readable collection address
YES	Nearby discovery	Address or GPS position, category/radius filters, distance result
YES	PostGIS	Geography point, GiST index, IsWithinDistance query
YES	Rental request	Future dates, inclusive price, availability calendar, overlap prevention
YES	Rental workflow	Relationship-aware actions, cancellation, incoming/outgoing lists
YES	Verified reviews	Requester only, completed rental only, one review per rental
YES	MVVM	Observable ViewModels, RelayCommand, binding-first XAML
YES	Repository / Unit of Work	Generic plus item, rental, and review repositories
YES	Service Layer	Client services and API business services
YES	State Pattern	Eight state classes including Cancelled, with transition tests
YES	Overdue detection	Hosted worker moves expired records to Overdue
YES	Security / tracing	Headers, correlation IDs, request logging, production secret inputs
YES	Testing	89/89 tests; 87.4% line coverage; HTTP/PostGIS integration
YES	CI/CD	Green PostGIS test/coverage job and signed Android APK job
YES	Docker / Dev Container	Compose health checks and .NET 10 Noble environment
YES	Accessibility	Semantic descriptions on important MAUI controls
NO	MediatR / CQRS Lite	Optional; excluded to keep the system compact and explainable
NO	SonarCloud	Optional; coverage artifacts are implemented instead

Scope decision: MediatR and SonarCloud are optional in the brief. The implementation prioritises a defensible State Pattern, real PostGIS testing, Android workflow, security controls, and reproducible delivery.

3.1 User-facing feature evidence

Eight genuine Android emulator captures show the principal user-facing workflows. Each claim is also traced to its XAML, ViewModel, service, and executable test evidence, so the report combines visible proof with maintainable-code evidence.

Feature	Visible behaviour	Traceable evidence
Login / registration	One account can list and request items; validation, bounded retry, and sign-out are available.	Views/LoginPage.xaml; LoginViewModel.cs; RentalApiTests; LoginViewModelTests
Browse marketplace	Search, category filter, sort order, paging, rating, and review count.	Views/ItemsListPage.xaml; ItemsListViewModel.cs; ItemsListViewModelTests
Item details	Rate, owner, address, availability, reviews, dates, and valid relationship-aware actions.	Views/ItemDetailPage.xaml; ItemDetailViewModel.cs; ItemDetailViewModelTests
List an item	Title, description, category, daily rate, typed address lookup, or current GPS.	Views/CreateItemPage.xaml; CreateItemViewModel.cs; CreateItemViewModelTests
My Listings	Current user's listings can be reviewed and edited without a separate account role.	Views/MyListingsPage.xaml; MyListingsViewModel.cs; MyListingsViewModelTests
Accessibility	Important inputs, actions, and status elements expose semantic descriptions.	MAUI XAML SemanticProperties; CI Android build
Security boundary The MAUI client hides invalid actions for usability, but ASP.NET Core services repeat ownership, requester, workflow, and validation checks. A modified client therefore cannot bypass the business rules.		

3.2 Spatial, workflow, and review evidence

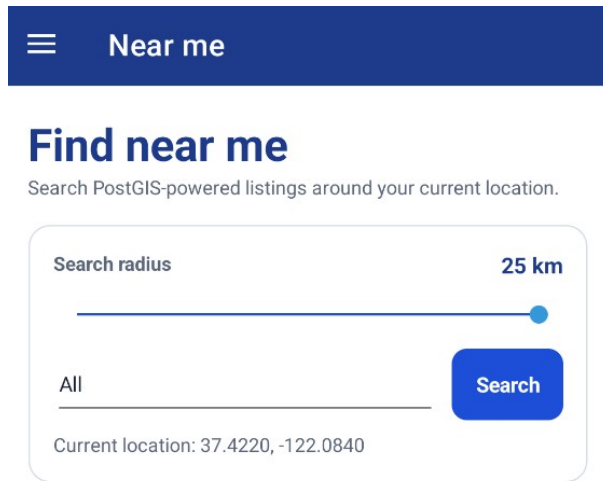
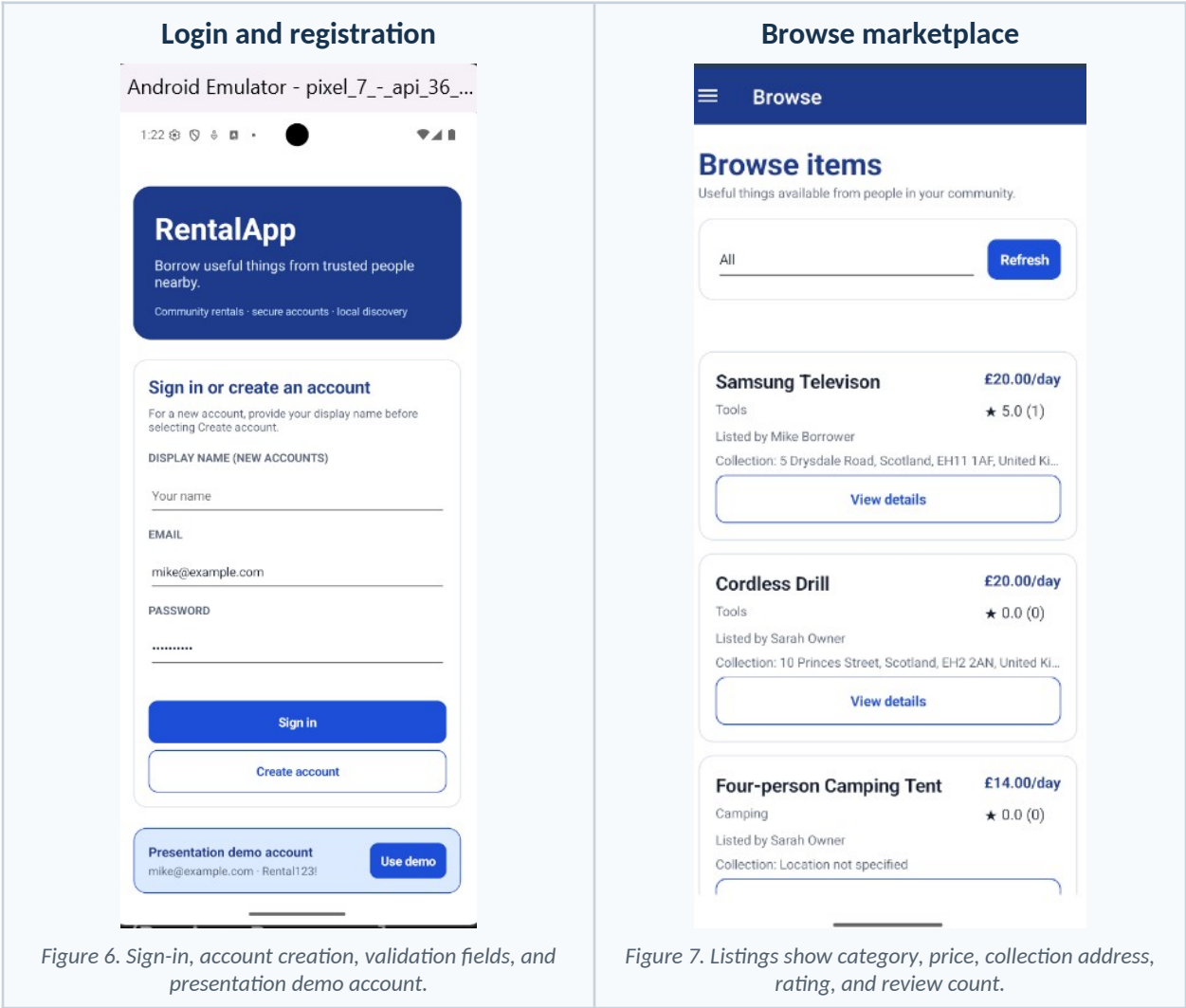


Figure 5. Real Android emulator capture of the PostGIS-powered Near me screen.

Feature	Implementation and test evidence
Near me	NearbyItemsPage.xaml -> NearbyItemsViewModel -> ItemService -> GET /items/nearby -> LocationService/ItemRepository -> PostGIS. LocationServiceTests, NearbyItemsViewModelTests, and ItemRepositoryTests verify the path.
Rental workflow	RentalsPage.xaml presents only valid creator/requester actions. RentalWorkflowService and eight State classes enforce request, approve, reject, cancel, start, overdue, return, and completion rules. Fourteen state-machine cases and eight workflow-service cases cover them.
Verified reviews	ReviewsPage.xaml offers eligible completed rentals. ReviewService rejects the wrong requester, an incomplete rental, duplicate submissions, and ratings outside 1-5. Review counts flow through listing and detail DTOs.
Demonstration route Use Mike to request Sarah's item; Sarah approves and starts; Mike returns it; Sarah completes it; Mike submits the verified review. This proves both sides of one transaction without separate account types.	

3.3 Authentication and catalogue evidence

These genuine emulator captures show the entry point and the populated marketplace. The demo account accelerates assessment without exposing a production credential.



3.4 Listing detail and creation evidence

The detail view exposes ownership-aware actions, while the creation form gathers the data required by the API and PostGIS location workflow.

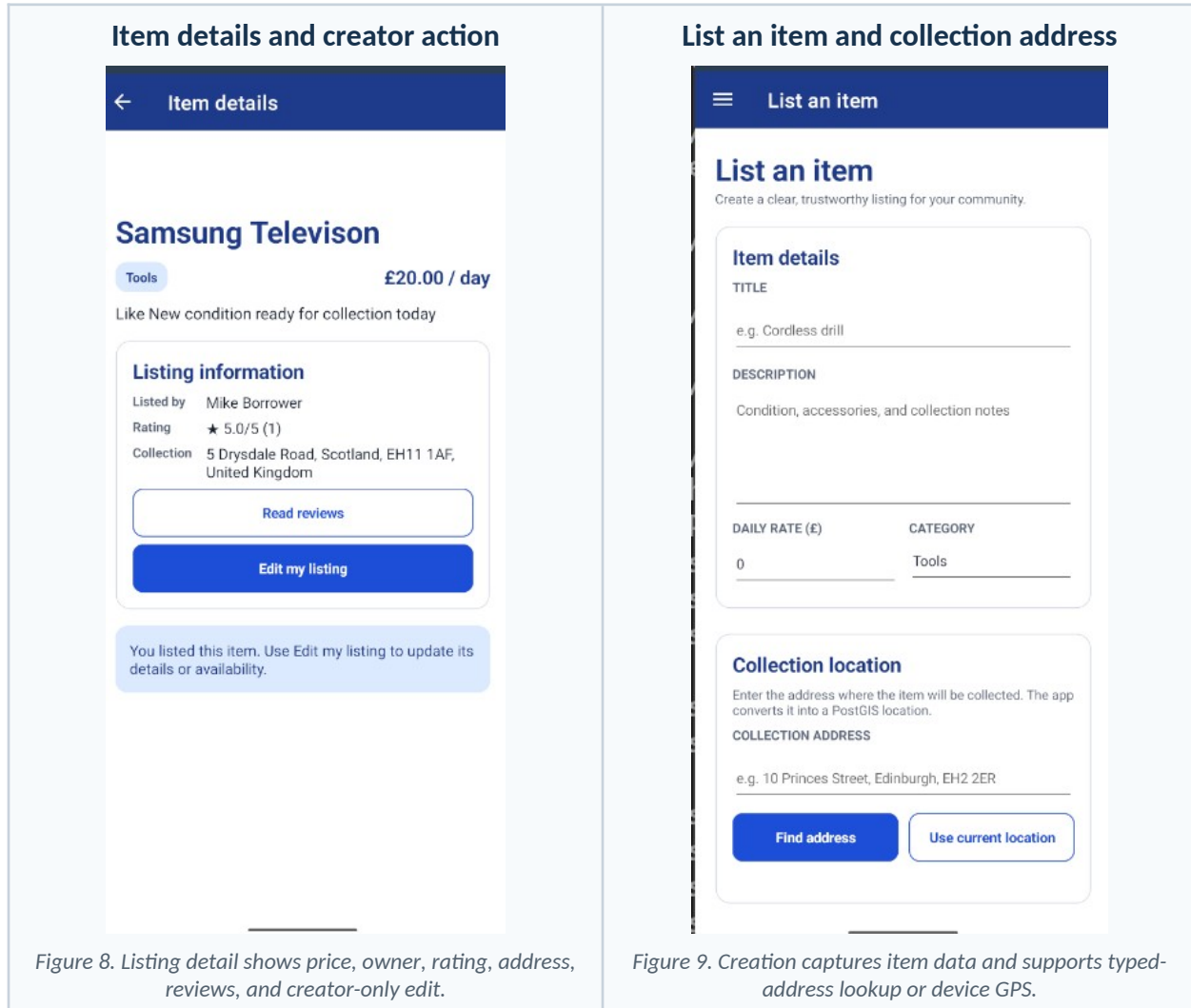


Figure 8. Listing detail shows price, owner, rating, address, reviews, and creator-only edit.

Figure 9. Creation captures item data and supports typed-address lookup or device GPS.

3.5 Workflow and account evidence

Genuine emulator captures show rental states, nearby search, navigation, reputation, unified account access, and sign-out.

Rental workflow

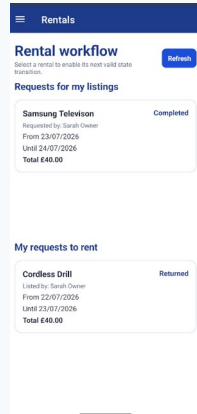


Figure 10. Completed and returned rental states.

Near Me

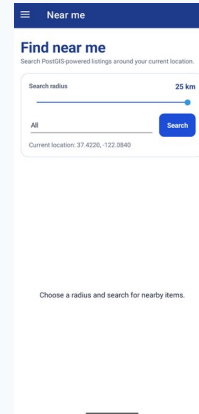


Figure 11. PostGIS radius and location controls.

Navigation and sign-out

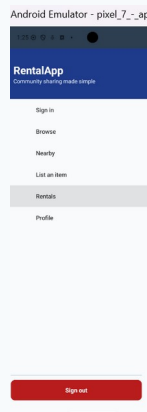


Figure 12. Shell navigation and persistent sign-out.

Profile and reputation

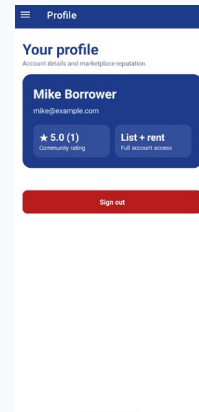


Figure 13. Reputation, unified access, and sign-out.

4. Testing documentation

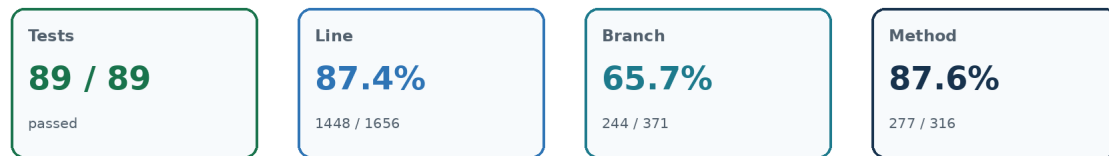
4.1 Verified result and coverage

Verified GitHub Actions run - 28 July 2026

89 total | 89 passed | 0 failed | 0 skipped. Cobertura line coverage: 87.4%.

Verified test and coverage export

GitHub Actions run #5 | 28 July 2026 | commit e047977



Handwritten assembly line coverage



Source: test-results-and-coverage artifact (tests.trx and CoverageReport/Cobertura.xml).

Figure 14. Coverage values rendered from the final CI tests.trx and Cobertura export.

Test group	Cases	Primary concern
RentalApiTests	7	Authenticated HTTP workflows against PostgreSQL/PostGIS
ItemRepositoryTests	1	Real PostgreSQL/PostGIS radius integration
Client / API services	11	HTTP, refresh, authentication, item, rental, and review routes
API application services	30	Validation, ownership, price, overlap, spatial, review, JWT, overdue
RentalStateMachineTests	14	Valid and invalid transitions including cancellation
ViewModels (9 classes)	26	Busy/error state, commands, paging, listings, reviews, profile

Total: 89 executed cases across 21 test classes. Theory and InlineData rows are counted as executed xUnit cases. The workflow enforces an 80.0% line-coverage gate.

4.2 Representative test excerpts

Example 1 - parameterised State Pattern test

```
[Theory]
[InlineData(RentalStatus.Requested, RentalStatus.Approved)]
[InlineData(RentalStatus.OutForRent, RentalStatus.Overdue)]
[InlineData(RentalStatus.Returned, RentalStatus.Completed)]
public void EnsureValidTransition_AllowedTransition_DoesNotThrow(
    RentalStatus current, RentalStatus next)
{
    var exception = Record.Exception(
        () => _machine.EnsureValidTransition(current, next));
    Assert.Null(exception);
}
```

Example 2 - business rule test

```
[Fact]
public async Task RequestAsync_OverlappingDates_RejectsSecondRequest()
{
    var data = await SeedUsersAndItemAsync(context);
    var start = DateTimeOffset.UtcNow.Date.AddDays(4);
    await service.RequestAsync(data.Borrower.Id,
        new(data.Item.Id, start, start.AddDays(2)));

    await Assert.ThrowsAsync<BusinessRuleException>(() =>
        service.RequestAsync(data.SecondBorrower.Id,
            new(data.Item.Id, start.AddDays(1), start.AddDays(3))));
}
```

Example 3 - real PostGIS integration test

```
[Fact]
public async Task GetNearbyAsync_ItemsInsideAndOutsideRadius_ReturnsOnlyNearbyItem()
{
    var results = await repository.GetNearbyAsync(
        55.9533, -3.1883, 2, ItemCategory.Tools);

    Assert.Contains(results, x => x.Item.Id == nearby.Id);
    Assert.DoesNotContain(results, x => x.Item.Id == distant.Id);
    Assert.All(results, x => Assert.InRange(x.DistanceMetres, 0, 2_000));
}
```

The tests follow Arrange-Act-Assert, use descriptive names, and combine fast in-memory/mock tests with one real spatial database fixture where an in-memory substitute would not prove translation to PostGIS.

5. CI/CD evidence

The workflow runs manually and on pushes or pull requests to main/master. The backend job starts PostgreSQL 16 with PostGIS, audits dependencies, restores, builds, executes tests, enforces the 80% line-coverage gate, creates readable reports, and uploads evidence. The Android job installs Java, .NET MAUI, and API 36, then creates an ephemeral coursework key and publishes a signed Release APK.

`.github/workflows/build.yml` (abridged)

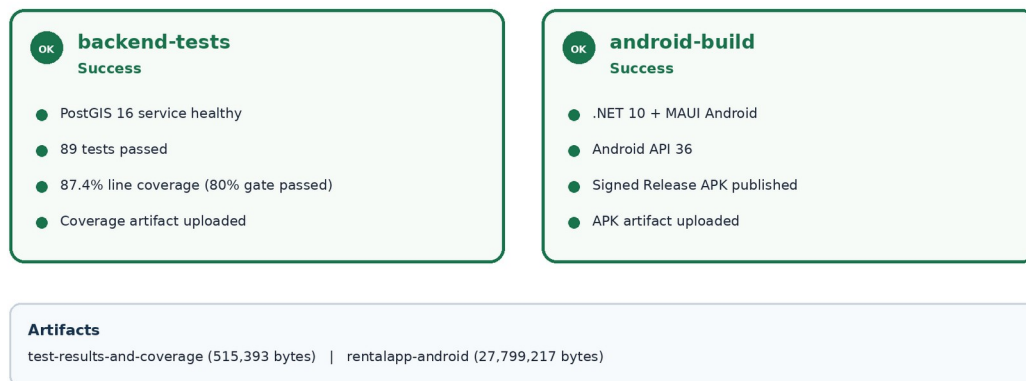
```
on:
  workflow_dispatch:
  push:
    branches: [main, master]
  pull_request:
    branches: [main, master]

jobs:
  backend-tests:
    services:
      postgres:
        image: postgres/postgis:16-3.5-alpine
    steps:
      - run: dotnet test RentalApp.Test/RentalApp.Test.csproj ...
      - run: python coverage gate (minimum 80%)
      - uses: actions/upload-artifact@v7

  android-build:
    steps:
      - uses: actions/setup-dotnet@v6
      - with: { workloads: maui-android }
      - run: sdkmanager platforms;android-36
      - run: dotnet publish ... -f net10.0-android
      - uses: actions/upload-artifact@v7
```

GitHub Actions - verified successful run

Build, test and package | run #5 | pull request #2 | commit e047977



Source: GitHub Actions run 30390459728.

Figure 15. API-verified summary of GitHub Actions run #5; both required jobs succeeded.

Evidence links: [successful workflow run](#) | [merged final pull request #2](#)

6. Design patterns implementation

6.1 MVVM

Views define layout and bind to observable state. ViewModels expose commands and depend on abstractions, so catalogue behaviour can be tested without a page or emulator. Generated ObservableProperty and RelayCommand members reduce repetitive notification code.

MVVM example - ItemsListViewModel

```
public partial class ItemsListViewModel(
    ItemService items, INavigationService navigation) : ViewModelBase
{
    public ObservableCollection<ItemSummaryDto> Items { get; } = [];

    [RelayCommand]
    private Task LoadAsync() => LoadPageAsync(reset: true);

    private Task LoadPageAsync(bool reset) => RunBusyAsync(async () =>
    {
        ItemCategory? category = Enum.TryParse<ItemCategory>(
            SelectedCategory, out var parsed) ? parsed : null;
        var result = await items.GetAllAsync(
            category, SearchTerm, SelectedSortOrder, _currentPage, PageSize);
        if (reset) Items.Clear();
        foreach (var item in result.Items) Items.Add(item);
        TotalResults = result.TotalCount;
        HasMoreItems = result.HasNextPage;
    });
}
```

6.2 Repository and Unit of Work

Repositories centralise EF Core expressions and eager-loading choices. The Item repository is the only layer that knows the spatial predicate. Unit of Work provides one explicit SaveChanges boundary for service operations.

Repository example - PostGIS radius query

```
var origin = new Point(longitude, latitude) { SRID = 4326 };
var radiusMetres = radiusKilometres * 1_000;

var results = await Context.Items
    .AsNoTracking()
    .Where(item => item.IsAvailable &&
        item.Location.IsWithinDistance(origin, radiusMetres))
    .Select(item => new {
        Item = item,
        DistanceMetres = item.Location.Distance(origin)
    })
    .OrderBy(result => result.DistanceMetres)
    .ToListAsync(cancellationToken);
```


6.3 Service Layer

RentalWorkflowService is the authoritative use-case layer. It coordinates repositories, validates rules, calculates inclusive price, checks actor permissions, invokes the state machine, and commits. Endpoints remain thin and ViewModels cannot bypass these rules.

Service example - validation and inclusive pricing

```
if (item.OwnerId == borrowerId)
    throw new BusinessException("You cannot rent your own item.");

if (await rentals.HasDateOverlapAsync(item.Id, start, end, cancellationToken))
    throw new BusinessException("The item is already booked for those dates.");

var numberOfDays = (end.Date - start.Date).Days + 1;
var rental = new Rental {
    ItemId = item.Id,
    BorrowerId = borrowerId,
    StartDateUtc = start,
    EndDateUtc = end,
    TotalPrice = item.DailyRate * numberOfDays
};
```

6.4 State Pattern (advanced)

A state object owns the allowed outgoing transitions for one RentalStatus. Adding a state or transition is localised instead of extending a large conditional. RentalWorkflowService separately checks listing-creator/requester permissions, preserving single responsibility.

Advanced pattern example - explicit state classes

```
public abstract class RentalState : IRentalState
{
    private readonly HashSet<RentalStatus> _allowed;

    protected RentalState(params RentalStatus[] allowed)
    {
        _allowed = new HashSet<RentalStatus>(allowed);
    }

    public abstract RentalStatus Status { get; }
    public bool CanTransitionTo(RentalStatus next)
    {
        return _allowed.Contains(next);
    }
}

public sealed class OutForRentState : RentalState
{
    public OutForRentState()
        : base(RentalStatus.Overdue, RentalStatus.Returned) { }

    public override RentalStatus Status => RentalStatus.OutForRent;
}
```

7. AI tool usage

Tools used: ChatGPT Codex for architecture, implementation support, refactoring, test design, debugging, and documentation review; Visual Studio Code C# Dev Kit for compiler diagnostics, navigation, and debugging.

Interaction	Suggestion	Evaluation / modification	Validation
Architecture	Separate MAUI, application, API, data, migrations, and test projects.	Accepted for testability and security. Rejected optional MediatR/SonarCloud to control scope.	Build, dependency, service, and integration tests.
Spatial search	Use SRID 4326 Point, geography column, GiST, and IsWithinDistance.	Checked X=longitude, Y=latitude; validated coordinate/radius bounds.	Real PostGIS fixture returns near item only.
Rental workflow	One class per state; keep role checks in the service.	Accepted to demonstrate Open/Closed design; overlap and inclusive-price rules retained.	Theory tests plus price, overlap, role, and overdue tests.
Android debugging	Use Logcat to diagnose splash exit and details crash.	Changes tied to observed stack traces: embed assemblies, correct App initialisation, harden route.	Clean rebuild/install and focused ViewModel tests.
Address entry	Forward-geocode typed addresses and reverse-geocode GPS.	Store readable address plus validated PostGIS point; remove raw coordinate fields from UI.	ViewModel and authenticated API workflow tests.
Unified accounts and review UX	Derive actions from listing-creator/requester relationships and carry review counts through DTOs.	Kept every API ownership and requester check; the UI is helpful but is not the security boundary.	Final regression: 89/89 tests and 87.4% coverage.
Submission upgrades	Add search/paging, cancellation, availability, rate limiting, readiness, tracing, security headers, and semantics.	Implemented as 13 focused commits and reviewed through pull request #1; corrected final CI failures before merge.	Both GitHub Actions jobs green; artifacts inspected.

Reflection

AI accelerated repetitive implementation and exposed alternatives, but it did not replace engineering judgement. Suggestions were narrowed to an explainable scope, compiled with warnings as errors, and tested at the appropriate boundary. Database and Android defects were diagnosed from evidence instead of accepting speculative changes. In the presentation, the student must explain every excerpt above and the trade-offs behind it.

8. References

1. [Microsoft \(2026\). .NET MAUI documentation](#)
2. [Microsoft \(2025\). Spatial data in Entity Framework Core](#)
3. [Microsoft \(2025\). Configure JWT bearer authentication in ASP.NET Core](#)
4. [PostGIS Project. ST_DWithin reference](#)
5. [NetTopologySuite project, API documentation](#)
6. [xUnit.net, Getting started with xUnit.net v3](#)
7. [Docker, Docker Compose documentation](#)
8. [GitHub, Building and testing .NET](#)

Submission verification

Check	Verified evidence
Public repository	https://github.com/justinwylie033/RentalApp_40535392 is public; master contains at least 20 reachable commits.
Required features	Feature matrix, source paths, architecture diagrams, and executable tests are included.
Tests / coverage	89/89 passed; 87.4% line coverage; 80% CI gate passed.
CI/CD	Run #5 succeeded for backend-tests and android-build; both evidence artifacts exist.
README / setup	Windows startup, Docker, Android, testing, deployment, and demo workflow are documented.
Credentials	No production secret is committed; local demo values are labelled and production Compose requires environment inputs.
Code explanation	Architecture guide, demonstration guide, XML summaries, and presentation-point comments are included.
AI disclosure	Tools, representative interactions, evaluation, modifications, and validation are documented above and in docs/AI_USAGE.md.
App screenshots	Eight genuine emulator captures document authentication, browsing, details, creation, spatial search, workflow, navigation, and profile.
Report format	Final DOCX and PDF use 12 pt body text, page numbers, all required sections, and fewer than 20 pages.
Final status Repository, application, architecture, tests, coverage, CI/CD, commit history, patterns, AI usage, and eight genuine emulator captures are traceable. The final Android workflow was demonstrated from the submitted source and is represented by the emulator evidence in this report.	